



Práctica Banco Nexus (Etapa 2)

≡ Materia	Sistemas Distribuidos
≡ Etiquetas	actividad uabcs
🕒 Creado	May 17, 2026 9:28 PM



Ariel Felixmarco Perez Iglesias
Omar Armando Urquidez Castro
Jose Ernesto Guluarte Frias
Jose Carlos Victorio Coronado

Etapa 2: Sincronización Simulada entre Sucursales

Implementación actualizada a Mongoose para estructurar modelos, conexión y operaciones CRUD del backend.

Resumen de implementación

La etapa 2 de Banco Nexus quedó extendida para rastrear el origen de cada operación bancaria por sucursal remota. Las rutas de depósito y retiro ahora aceptan `sucursal`, la persistencia de transacciones guarda ese dato, el frontend permite elegir la sucursal operativa y el historial reciente la muestra de forma visible.

Además, se agregaron scripts de simulación concurrente para ejecutar operaciones paralelas desde cinco sucursales (`CDMX`, `GDL`, `MTY`, `PUE`, `TIJ`) sobre una misma cuenta y analizar si aparecen colisiones o inconsistencias en el saldo final.

En la versión actual del backend, la migración a Mongoose quedó separada así:

- `backend/server.js` : punto de arranque del servidor.
- `backend/src/app.js` : definición de rutas y composición de Express con queries de Mongoose.
- `backend/src/db.js` : conexión reutilizable con `mongoose.connect(...)`.
- `backend/src/models.js` : esquemas y modelos de Mongoose para clientes, cuentas y transacciones.
- `backend/transactions.js` : lógica compartida de validación, normalización y registro de operaciones usando modelos Mongoose.
- `backend/tests/transaction.test.js` : pruebas unitarias del flujo bancario.

Estructura de datos

Los modelos del backend ahora se definen con Mongoose:

- `Client` : colección `clientes`
- `Account` : colección `cuentas`
- `Transaction` : colección `transacciones`

La cuenta se sigue relacionando con el cliente mediante `cliente = curp`, y las transacciones se siguen vinculando por `cuenta`.

Estructura de transacción

Cada transacción queda registrada con la siguiente forma:

```
{
  "cuenta": "10000000001",
  "tipo": "deposito",
  "monto": 150.25,
  "saldoResultante": 18900.7,
  "fecha": "2026-05-17T00:00:00.000Z",
  "sucursal": "GDL",
  "descripcion": "Depósito"
}
```

Reglas aplicadas:

- Si `sucursal` no llega en el request, se normaliza a `CDMX`.
- Las transacciones históricas del seed también se normalizan a `CDMX`.
- Los valores válidos para `sucursal` son `CDMX`, `GDL`, `MTY`, `PUE` y `TIJ`.
- Las operaciones CRUD de cuentas y transacciones se ejecutan con modelos Mongoose en lugar del driver nativo.

Endpoints involucrados

POST /api/deposito

Body esperado:

```
{
  "cuenta": "10000000001",
  "monto": 150.25,
  "sucursal": "GDL"
}
```

POST /api/retiro

Body esperado:

```
{
  "cuenta": "10000000001",
  "monto": 50,
  "sucursal": "MTY"
}
```

GET /api/cuenta/:cuenta

Devuelve la cuenta y las últimas transacciones incluyendo `sucursal`.

GET /api/historial/:cuenta

Devuelve el historial completo de la cuenta incluyendo `sucursal`.

Scripts agregados

Ubicación: `backend/scripts/`

- `branchOperation.js`: helper compartido para operaciones remotas y simulación.
- `operacionSucursalCDMX.js`
- `operacionSucursalGDL.js`
- `operacionSucursalMTY.js`
- `operacionSucursalPUE.js`
- `operacionSucursalTIJ.js`
- `simulateConcurrentBranches.js`

Scripts de `package.json`:

- `npm run init-db`
- `npm run start`
- `npm run dev`
- `npm test`
- `npm run simulate-branches`
- `npm run simulate-cdmx`
- `npm run simulate-gdl`
- `npm run simulate-mty`
- `npm run simulate-pue`
- `npm run simulate-tij`

Comandos de ejecución

1. Levantar MongoDB

Si ya existe el contenedor:

```
docker start banco_nexus_db
```

2. Sembrar la base

```
cd backend  
npm run init-db
```

3. Levantar backend

```
cd backend  
npm run start
```

4. Levantar frontend

```
cd frontend  
npm run dev
```

5. Correr simulación concurrente

```
cd backend  
ACCOUNT=1000000002 RUNS=3 npm run simulate-branches
```

6. Ejecutar pruebas del backend

```
cd backend  
npm test
```

Escenarios probados

1. Seed limpio con sucursal por defecto

Se verificó que la cuenta `1000000001` expone transacciones históricas con `sucursal: "CDMX"` en la API.

Resultado observado:

- Saldo inicial: `18750.45`
- Primera sucursal en transacciones recientes: `CDMX`
- Transacciones recientes devueltas: `4`

ÚLTIMAS TRANSACCIONES					
FECHA	TIPO	DESCRIPCIÓN	SUCURSAL	MONTO	SALDO RESULTANTE
22/4/2024	deposito	Transferencia recibida	CDMX	+\$2,500.00	\$18,750.45
4/3/2024	retiro	Retiro en cajero	CDMX	-\$1,250.00	\$16,250.45
10/2/2024	deposito	Depósito en ventanilla	CDMX	+\$5,500.45	\$17,500.45
15/1/2024	deposito	Depósito inicial	CDMX	+\$12,000.00	\$12,000.00

2. Depósito válido desde sucursal remota

Request probado:

```
{
  "cuenta": "10000000001",
  "monto": 150.25,
  "sucursal": "GDL"
}
```

Resultado observado:

- HTTP 200
- saldoAnterior : 18750.45
- nuevoSaldo : 18900.7
- sucursal : GDL

ÚLTIMAS TRANSACCIONES					
FECHA	TIPO	DESCRIPCIÓN	SUCURSAL	MONTO	SALDO RESULTANTE
17/5/2026	deposito	Depósito	GDL	+\$150.25	\$18,900.70
22/4/2024	deposito	Transferencia recibida	CDMX	+\$2,500.00	\$18,750.45
4/3/2024	retiro	Retiro en cajero	CDMX	-\$1,250.00	\$16,250.45
10/2/2024	deposito	Depósito en ventanilla	CDMX	+\$5,500.45	\$17,500.45
15/1/2024	deposito	Depósito inicial	CDMX	+\$12,000.00	\$12,000.00

3. Retiro válido desde sucursal remota

Request probado:

```
{
  "cuenta": "10000000001",
  "monto": 50,
  "sucursal": "MTY"
}
```

Resultado observado:

- HTTP 200
- saldoAnterior : 18900.7
- nuevoSaldo : 18850.7
- sucursal : MTY

ÚLTIMAS TRANSACCIONES					
FECHA	TIPO	DESCRIPCIÓN	SUCURSAL	MONTO	SALDO RESULTANTE
17/5/2026	retiro	Retiro en cajero	MTY	-\$50.00	\$18,850.70
17/5/2026	deposito	Depósito	GDL	+\$150.25	\$18,900.70
22/4/2024	deposito	Transferencia recibida	CDMX	+\$2,500.00	\$18,750.45
4/3/2024	retiro	Retiro en cajero	CDMX	-\$1,250.00	\$16,250.45
10/2/2024	deposito	Depósito en ventanilla	CDMX	+\$5,500.45	\$17,500.45
15/1/2024	deposito	Depósito inicial	CDMX	+\$12,000.00	\$12,000.00

4. Retiro rechazado por saldo insuficiente

Request probado:

```
{
  "cuenta": "10000000011",
  "monto": 999999,
  "sucursal": "TIJ"
}
```

Resultado observado:

- HTTP 400
- Error: Saldo insuficiente

5. Verificación visual del frontend

Se confirmó en la interfaz local:

- Selector **Operar desde** con las cinco sucursales.
- Tabla de transacciones con columna **Sucursal**.
- Transacciones nuevas reflejadas con **GDL** y **MTY**.

Resultados de simulación concurrente

Cuenta probada: **1000000002**

Operaciones concurrentes por corrida:

- **CDMX** : depósito de **120.00**
- **GDL** : depósito de **85.50**
- **MTY** : depósito de **60.25**
- **PUE** : depósito de **40.75**
- **TIJ** : depósito de **95.00**

Suma esperada por corrida: **401.50**

Corrida 1

- Saldo inicial: **32400.00**
- Saldo esperado: **32801.50**
- Saldo observado: **32580.25**
- Diferencia: **221.25**
- Operaciones exitosas: **5**
- Operaciones fallidas: **0**
- Transacciones nuevas registradas: **5**
- Inconsistencia detectada: **sí**

Corrida 2

- Saldo inicial: 32580.25
- Saldo esperado: 32981.75
- Saldo observado: 32716.00
- Diferencia: 265.75
- Operaciones exitosas: 5
- Operaciones fallidas: 0
- Transacciones nuevas registradas: 5
- Inconsistencia detectada: sí

Corrida 3

- Saldo inicial: 32716.00
- Saldo esperado: 33117.50
- Saldo observado: 32811.00
- Diferencia: 306.50
- Operaciones exitosas: 5
- Operaciones fallidas: 0
- Transacciones nuevas registradas: 5
- Inconsistencia detectada: sí

Conclusión

La simulación entre sucursales fue exitosa y permitió observar un problema clásico de concurrencia: todas las operaciones HTTP respondieron como exitosas y todas las transacciones se insertaron, pero el saldo final no coincidió con la suma esperada en ninguna de las tres corridas.

Esto confirma que la lógica actual de lectura-modificación-escritura del saldo es vulnerable a condiciones de carrera cuando varias sucursales operan en paralelo sobre la misma cuenta. Para esta etapa no se corrigió ese comportamiento a propósito; se dejó visible y documentado como evidencia de la sincronización simulada entre sucursales.